

LG-aimers

EXAONE-4.0-1.2B 모델에 대한 Quantization 및 성능 개선 작업 진행

- EXAONE의 small scale 모델인 1.2B 모델.
- 단순 파라미터 수를 줄이면 메모리와 속도는 빨라지나 정확도가 크게 열화되는 문제 발생
- 모델 크기를 줄이고 속도를 증가시키면서 정확도를 유지시키기 위한 경량화 기법 고안하기

1.2B 모델 기본

base_model
파일 폴더
C:\Users\wel071\Downloads\base_model
2.39GB (2,570,005,161 바이트)

- 2.39GB 용량 모델

Quantization 및 성능 개선 후

model3
파일 폴더
C:\Users\wel071\Downloads
1.30GB (1,401,840,526 바이트)

- 1.3GB로 용량 감소

단순 Quantization 모델 성능 (manta-dataset 활용) Quantization 및 성능 개선 후 (데이터 셋 조합)

Tasks	Version	Filter	n-shot	Metric	Value	Stderr
gsm8k	3	flexible-extract	5	exact_match	0.5977	0.0217
		strict-match	5	exact_match	0.4727	0.0221

- gsm8k task - 0.5977

Tasks	Version	Filter	n-shot	Metric	Value	Stderr
gsm8k	3	flexible-extract	5	exact_match	0.6699	0.0208
		strict-match	5	exact_match	0.6582	0.0210

- gsm8k task - 0.6699

Groups	Version	Filter	n-shot	Metric	Value	Stderr
mmlu_pro	2	custom-extract	5	exact_match	0.3646	0.0057

Groups	Version	Filter	n-shot	Metric	Value	Stderr
mmlu_pro	2	custom-extract	5	exact_match	0.3792	0.0057

- mmlu_pro task - 0.3646

- mmlu_pro task - 0.3792

Tasks	Version	Filter	n-shot	Metric	Value	Stderr
hellaswag	1	none	0	acc	↑ 0.3926 ± 0.0216	
hellaswag	1	none	0	acc_norm	↑ 0.3984 ± 0.0217	

- hellaswag task - 0.3926

- hellaswag task - 0.3984

Tasks	Version	Filter	n-shot	Metric	Value	Stderr
kobest	1	none	0	acc	↑ 0.4945 ± 0.0099	
kobest	1	none	0	acc_norm	↑ 0.5113 ± 0.0099	

- kobest task - 0.4945

- kobest task - 0.5113

Tasks	Version	Filter	n-shot	Metric	Value	Stderr
gsm8k_llama	3	flexible_extract	8	exact_match	↑ 0.7168 ± 0.0199	
gsm8k_llama	3	strict_match	8	exact_match	↑ 0.7207 ± 0.0198	

- gsm8k_llama task - 0.7168

- gsm8k_llama task - 0.7207

Groups	Version	Filter	n-shot	Metric	Value	Stderr
mmlu_llama	1	strict_match	1	exact_match	0.5121 ± 0.0044	
mmlu_llama	1	strict_match	1	exact_match	0.5148 ± 0.0044	

- mmlu_llama task - 0.5121

- mmlu_llama task - 0.5148

Quantization + 성능개선 코드

```
#필수 라이브러리 및 버전 확인
datasets==4.4.1
transformers==4.57.3
tokenizers==0.22.1
accelerate==1.10.1
datasets==4.4.1
huggingface-hub==0.36.0
vllm==0.14.1
llmcompressor
```

```
typing_extensions
peft
```

```
from datasets import load_dataset, concatenate_datasets, Dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from llmcompressor import oneshot
from llmcompressor.modifiers.quantization import GPTQModifier
import os
import torch
import json
import gc

# 1. 환경 설정 및 메모리 정리
os.environ["TOKENIZER_PARALLELISM"] = "false"
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"

torch.cuda.empty_cache()
gc.collect()

MODEL_ID = "LGAI-EXAONE/EXAONE-4.0-1.2B"

# 2. 모델 및 토크나이저 로드
print(f"Loading Model: {MODEL_ID}")
model = AutoModelForCausalLM.from_pretrained(MODEL_ID, torch_dtype=torch.bfloat16, device_map="auto")
tokenizer = AutoTokenizer.from_pretrained(MODEL_ID, trust_remote_code=True)

# 3. 데이터셋 로드 (Total 420 Samples)
print("Loading datasets...")

# (1) MANTA (한국어/일반) - 100개
# 스트리밍으로 충분히 가져온 후 셔플링
ds_manta_stream = load_dataset("LGAI-EXAONE/MANTA-1M", split=
```

```

t="train", streaming=True)
ds_manta = Dataset.from_list(list(ds_manta_stream.take(100
0))).shuffle(seed=42).select(range(100))

# (2) GSM8K (기초 수학) - 40개
ds_gsm8k = load_dataset("gsm8k", "main", split="train").sel
ect(range(40))

# (3) OpenR1-Math (심화 추론) - 180개
try:
    print("Loading OpenR1-Math-220k...")
    ds_r1_stream = load_dataset("open-r1/OpenR1-Math-220k",
split="train", streaming=True)
    ds_r1 = Dataset.from_list(list(ds_r1_stream.take(100
0))).shuffle(seed=123).select(range(180))
except Exception as e:
    print(f"⚠ OpenR1 load failed: {e}. Fallback to GSM8K.")
    ds_r1 = load_dataset("gsm8k", "main", split="train").se
lect(range(40, 220))

# (4) [NEW] MMLU (다분야 지식) - 100개
# 다양한 분야를 커버하기 위해 주요 카테고리에서 20개씩 추출
print("Loading MMLU (Multi-task)...")
mmlu_subjects = [
    'high_school_mathematics', # 수학
    'college_physics',         # 과학
    'professional_law',        # 인문/사회
    'clinical_knowledge',       # 의학
    'world_religions'           # 문화/상식
]
ds_mmlu_list = []
for subject in mmlu_subjects:
    try:
        # MMLU는 보통 'test' 스플릿을 사용
        ds_sub = load_dataset("cais/mmlu", subject, split
="test", streaming=True)
        ds_mmlu_list.extend(list(ds_sub.take(20)))
    except Exception as e:

```

```
print(f"Warning: Failed to load MMLU subject {subject}")
```

```
ds_mmlu = Dataset.from_list(ds_mmlu_list).shuffle(seed=42)
```

4. 전처리 함수 정의

```
def preprocess_general(messages):  
    return {"text": tokenizer.apply_chat_template(messages,  
add_generation_prompt=False, tokenize=False)}
```

```
def preprocess_manta(example):  
    return preprocess_general(example["conversations"])
```

```
def preprocess_gsm8k(example):  
    messages = [  
        {"role": "user", "content": example["question"]},  
        {"role": "assistant", "content": example["answer"]}  
    ]  
    return preprocess_general(messages)
```

```
def preprocess_openr1(example):  
    q = example.get("problem") or example.get("question")  
    # solution을 사용하여 추론 과정 학습 유도  
    a = example.get("solution") or example.get("answer")  
    messages = [  
        {"role": "user", "content": q},  
        {"role": "assistant", "content": a}  
    ]  
    return preprocess_general(messages)
```

```
def preprocess_mmlu(example):  
    # MMLU: 객관식 문제 -> Q&A 포맷으로 변환  
    # 0,1,2,3 -> A,B,C,D  
    options = ["A", "B", "C", "D"]  
    answer_idx = example["answer"]  
    answer_char = options[answer_idx]
```

```
# 질문과 보기 구성
```

```

        question_text = f"{example['question']}\nOptions:\nA)
{example['choices'][0]}\nB) {example['choices'][1]}\nC) {ex
ample['choices'][2]}\nD) {example['choices'][3]}"

        messages = [
            {"role": "user", "content": question_text},
            {"role": "assistant", "content": f"Answer: {answer_
char}"}]
        ]
        return preprocess_general(messages)

# 전처리 적용
print("Preprocessing datasets...")
ds_manta_proc = ds_manta.map(preprocess_manta, remove_colum
ns=ds_manta.column_names)
ds_gsm8k_proc = ds_gsm8k.map(preprocess_gsm8k, remove_colum
ns=ds_gsm8k.column_names)
ds_r1_proc = ds_r1.map(preprocess_openr1, remove_columns=ds
_r1.column_names)
ds_mmlu_proc = ds_mmlu.map(preprocess_mmlu, remove_columns=
ds_mmlu.column_names)

# 데이터 병합 (Total 420)
ds_combined = concatenate_datasets([ds_manta_proc, ds_gsm8k
_proc, ds_r1_proc, ds_mmlu_proc])
ds_combined = ds_combined.shuffle(seed=42)

print(f"Total calibration samples: {len(ds_combined)}")
print("Distribution: Manta(100) + GSM8K(40) + OpenR1(180) +
MMLU(100)")

```

```

# 5. 레시피 설정 (Sequential Targets 수정)
recipe = [
    GPTQModifier(
        targets=["Linear"],
        scheme="W4A16",
        ignore=["lm_head", "embed_tokens"], #양자화 시 성능
하락 큰 lm_head, embed_tokens는 대상 제거

```

```

        dampening_frac=0.01,
        sequential_targets=["ExaoneDecoderLayer"],
        block_size=128,
    )
]

```

6. 양자화 수행

```

print("Starting Quantization...")
oneshot(
    model=model,
    dataset=ds_combined,
    recipe=recipe,
    max_seq_length=2048, # 롱 컨텍스트 대응 위해 증가
    num_calibration_samples=len(ds_combined),
)

```

```

SAVE_DIR = "model3"

```

safetensors 저장

```

model.save_pretrained(SAVE_DIR, save_compressed=True, safe_
serialization=True)
tokenizer.save_pretrained(SAVE_DIR)

```

```

print(f"Model saved to {SAVE_DIR} with vLLM optimized confi
g.")

```